Fundamentals of Deep Learning and Programming

Lecture 3: Neural Network Basics (2)

Fall Semester 2026

Adjunct Professor: Donghoon Kang

2026 Calendar



- **UST Daejeon Tour 10.02 (Fri.)**
- **Holiday 10.09 (Fri.): Hangeul Day**
- **KIST sports festival: 10.16 (Fri.)**

2026 Calendar



Week 6 (11.06)

Mid-term Exam

Term Project

Must be performed alone

- Should select a paper published only in CVPR, ICCV, ECCV, or NeurIPS after 2022 (inclusive).
- Should write an 4 ~ 6 page report analyzing the "training code" of the selected paper.

Before Week 7 (10.16)

Should decide which paper to choose. (also possible to change another paper later)

Before Week 15 (12.11)

Should submit an 4 ~ 6 page report as a term-project.

Schedule

October

Week 4 (10.02)

10.02	No class: UST Daejeon Tour
10.09	No class: Holiday (Hangeul Day)
10.16	No class: KIST Sports Festival
Week 4 (10.23)	Neural Networks Training and CNN
Week 5 (10.30)	Fundamentals of Object Recognition

Schedule

November

Week 6 (11.06)	Mid-term Exam
Week 7 (11.13)	Fundamentals of Natural Language Processing
Week 8 (11.20)	Transformer (1): Self-Attention and Encoder
Week 9 (11.27)	Transformer (2): Pretraining and Fine-tuning

Schedule

December

Week 10 (12.04)	Diffusion Models
Week 11 (12.11)	Fundamentals of Video Recognition
Week 12 (12.18)	Final Exam

Using GPU

```
# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

a = torch.tensor([1., 2.], device=device)
b = torch.tensor([1., 2.])
print(a)
print(b)
```

```
tensor([1., 2.], device='cuda:0')
tensor([1., 2.])
```

c02_pytorch10.py



CPU



GPU

argparse (3)

```
import argparse

def main():
    # 1. Create parser
    parser = argparse.ArgumentParser(description="simple example")

    # 2. Add argument for number of iterations
    parser.add_argument("–my_iterations", type=int, default=5,
                        help="Number of times to run the loop (default: 5)")

    # 3. Parse arguments
    args = parser.parse_args()

    # 4. Use iterations in a for loop
    for i in range(args.my_iterations):
        print(f"Iteration {i+1}")

if __name__ == "__main__":
    main()
```

MNIST Dataset

70,000 grayscale images of handwritten digits, from 0 to 9

- Each image is a 28 × 28 grayscale square.
- Normalization:

$$pixel_{normalized} = \frac{pixel_{original}}{255.0}$$



0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	8	38	137	146	232	254	255	255	197	109	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	87	197	253	253	253	253	253	253	253	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	120	237	253	253	253	248	209	139	139	230	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	112	229	210	128	96	0	0	0	0	117	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	28	0	0	0	0	0	0	45	241	245	82	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	60	253	125	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	32	134	148	98	127	4	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	7	201	253	200	59	12	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	122	246	253	223	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	233	253	253	236	55	11	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	233	253	253	253	253	210	48	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	68	193	185	243	253	253	173	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	59	253	253	226	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	22	253	253	226	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	10	93	253	253	123	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	14	180	253	253	228	42	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	95	181	253	253	253	66	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	6	33	33	100	178	253	253	253	241	88	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	32	236	253	253	253	253	253	228	122	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	143	253	253	253	154	76	24	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

os.path.join() and os.makedirs

```python
import os

# Example: we want to work with a folder for MNIST dataset
base_dir = "."        # current directory
sub_dir = "data"
dataset = "MNIST"
raw_dir = "raw"

# os.path.join() combines parts into a valid path for your OS
# On Windows -> ".\\data\\MNIST\\raw"
# On Linux/Ubuntu/Mac -> "./data/MNIST/raw"
target_path = os.path.join(base_dir, sub_dir, dataset, raw_dir)
print("Joined path:", target_path)

# os.path.exists() checks whether the given path already exists
if os.path.exists(target_path):
    print("OK. The folder already exists:", target_path)
else:
    print("Not OK. The folder does NOT exist, creating it now...")
    os.makedirs(target_path, exist_ok=True)   # safe folder creation
    print("Folder created:", target_path)
```

```
Joined path: .\data\MNIST\raw
OK. The folder already exists: .\data\MNIST\raw
```

c04_path.py

os.path.join()

This function automatically uses the correct path separator for the operating system (a backslash \ on Windows and a forward slash / on Ubuntu). For example, os.path.join(".", "data") will correctly produce ./data on both systems, creating a valid path.

os.makedir()

This function creates a directory at the specified path. Using exist_ok=True is a standard practice that prevents the code from raising an error if the directory already exists.

How to download a file from the web

```python
import os, urllib.request

# Target folder
target_dir = os.path.join(".", "my_data", "MNIST", "raw")
os.makedirs(target_dir, exist_ok=True)

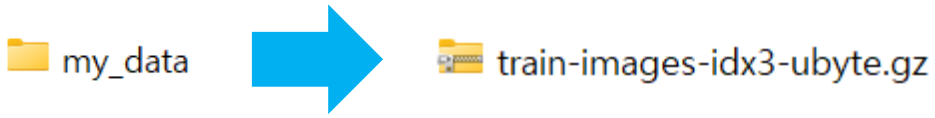
# One MNIST file (train images)
url = "https://storage.googleapis.com/cvdf-datasets/mnist/train-images-idx3-ubyte.gz"
out_path = os.path.join(target_dir, "train-images-idx3-ubyte.gz")

# Download file
urllib.request.urlretrieve(url, out_path)

print("OK. Download complete:", out_path)
```

```
OK. Download complete: .\my_data\MNIST\raw\train-images-idx3-ubyte.gz
```

c04_download.py



urllib.request()

This is a standard Python library for downloading files from the web.

The urllib.request.urlretrieve function performs an HTTP request and saves the file, working the same way regardless of the underlying operating system

Download MNIST dataset manually (URL for MNIST has changed)

```python
import os
import urllib.request

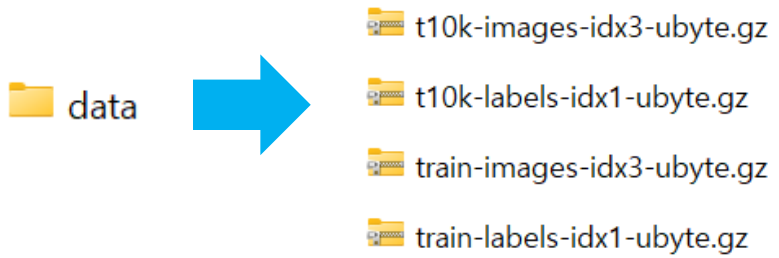
# Target directory
target_dir = os.path.join(".", "data", "MNIST", "raw")
os.makedirs(target_dir, exist_ok=True)

# Files to download (from Google Cloud mirror)
base_url = "https://storage.googleapis.com/cvdf-datasets/mnist/"
files = [
    "train-images-idx3-ubyte.gz",
    "train-labels-idx1-ubyte.gz",
    "t10k-images-idx3-ubyte.gz",
    "t10k-labels-idx1-ubyte.gz",
]

print(f"Downloading MNIST dataset into {target_dir} ...")

for fname in files:
    url = base_url + fname
    out_path = os.path.join(target_dir, fname)
    if not os.path.exists(out_path):
        print(f" -> {fname}")
        urllib.request.urlretrieve(url, out_path)
    else:
        print(f" [skip] {fname} already exists")

print("OK! MNIST dataset is ready in ./data/MNIST/raw/")
```




```
Downloading MNIST dataset into .\data\MNIST\raw ...
 -> train-images-idx3-ubyte.gz
 -> train-labels-idx1-ubyte.gz
 -> t10k-images-idx3-ubyte.gz
 -> t10k-labels-idx1-ubyte.gz
OK! MNIST dataset is ready in ./data/MNIST/raw/
```

download_mnist.py

Python

Special Methods (1)
(i.e., double underscore methods

```
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

# Vector2D usage
v1, v2 = Vector2D(2, 3), Vector2D(1, 1)

a = v1 + v2
print(a)
print(a.x, a.y)
```

```
<__main__.Vector2D object at 0x000001EB4E80FE50>
3 4
```

c04_special_method1.py

Python

Special Methods (2) : __repr__( )

```
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __repr__(self):  # developer-friendly representation
       return f"Vector2D({self.x}, {self.y})"


# Vector2D usage
v1, v2 = Vector2D(2, 3), Vector2D(1, 1)

a = v1 + v2
print(a)
```

```
Vector2D(3, 4)
```

c04_special_method1.py

Python

Special Methods (3): __str__( )

```
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __sub__(self, other):  # vector subtraction
        return Vector2D(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):  # scalar multiplication
        return Vector2D(self.x * scalar, self.y * scalar)

    def __matmul__(self, other):  # dot product
        return self.x * other.x + self.y * other.y

    def __str__(self):  # user-friendly string
        return f"({self.x}, {self.y})"

# Vector2D usage
v1, v2 = Vector2D(2, 3), Vector2D(1, 1)

b = v1 - v2
c = v1 * 2  # in this implementation, v1 * v2 causes error
d = v1 @ v2

print(b,c,d)
```

```
(1, 2) (4, 6) 5
```

c04_special_method1.py

Python

Special Methods (4): `__call__( )`

```
class Dog:
    def __call__(sef):
        print('mung mung')

my_dog = Dog()
my_dog()
```

```
mung mung
```

c04_call1.py

Objects can be called like functions via `__call__()` in the class.

```
class Cat:
    def __call__(self, num):
        print('ya-ong ' * num)

my_cat = Cat()
my_cat(3)
```

```
ya-ong ya-ong ya-ong
```

c04_call1.py

Python

Special Methods (5): `__enter__()` and `__exit__()`

```
class DummyContext:
    # Context manager using __enter__ and __exit__.
    def __enter__(self):
        print("Entering context")
        return "Resource Acquired"

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exiting context")

# The standard way to open a file using 'with'
with open('example.txt', 'w') as f:
    f.write("This is a test.")

# Context manager
with DummyContext() as res:
    print(res)
```

```
Entering context
Resource Acquired
Exiting context
```

Objects can be **used in a with block** via `__enter__()` and `__exit__()`.

c04_enter_exit_1.py

Python

Special Methods (6): __enter__() and __exit__()

```python
class MyContext:
    def __enter__(self):
        print("--- enter ---")
        return "Resource"

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("--- exit ---")
        print("exc_type:", exc_type)  # The exception class (e.g., ZeroDivisionError, ValueError).
        print("exc_val :", exc_val)   # The actual exception instance (e.g., ZeroDivisionError("division by zero")
        print("exc_tb  :", exc_tb)    # The traceback object (Can be used for debugging, logging, etc.)
        return True  # suppress exception if any

# Case 1: No exception
with MyContext() as res:
    print("Using:", res)

# Case 2: With exception
with MyContext() as res:
    print("About to divide by zero...")
    1 / 0   # ZeroDivisionError raised inside context
```

```
--- enter ---
Using: Resource
--- exit ---
exc_type: None
exc_val : None
exc_tb  : None
```

```
--- enter ---
About to divide by zero...
--- exit ---
exc_type: <class 'ZeroDivisionError'>
exc_val : division by zero
exc_tb  : <traceback object at 0x0000011375EF5700>
```

c04_enter_exit_2.py

Python

Special Methods (7): simple Dataset

```python
class SimpleDataset:
    # A simple dataset that stores a list of items.
    def __init__(self, data):
        self.data = data

    def __len__(self):  # length of dataset
        return len(self.data)

    def __getitem__(self, idx):  # indexing
        return self.data[idx]

    def __iter__(self):  # makes the class iterable
        self._i = 0
        return self

    def __next__(self):  # iteration logic
        if self._i < len(self.data):
            item = self.data[self._i]
            self._i += 1
            return item
        raise StopIteration

# Dataset usage
dataset = SimpleDataset([10, 20, 30])
print("Dataset length:", len(dataset))
print("Index 1:", dataset[1])
for item in dataset:
    print("Iter item:", item)
```

```
Dataset length: 3
Index 1: 20
Iter item: 10
Iter item: 20
Iter item: 30
```

c04_dataset_1.py

DataLoader

In PyTorch, a DataLoader helps you **load your dataset in small batches** (instead of all at once).

- It also lets you **shuffle** the data each epoch.
- It makes training faster and easier to manage.

Think of it like a "food delivery service"  :

- The **dataset** is the kitchen (all food available).
- The **DataLoader** is the **waiter**, bringing food to the table in small portions (batches).

Simple DataLoader (1)

```python
import torch
from torch.utils.data import Dataset, DataLoader

# 1. Define a simple Dataset class
class SimpleDataset(Dataset):
    def __init__(self, data):
        # data (list): Data as a list of numbers.
        self.data = data

    def __len__(self):
        # Return the total number of data points in the dataset
        return len(self.data)

    def __getitem__(self, idx):
        # Return the data point at the given index
        sample = self.data[idx]

        # Convert to a PyTorch tensor
        return torch.tensor(sample, dtype=torch.float32)

# 2. Prepare the data
# For this example, we use a list of numbers from 0 to 99.
my_data = list(range(100))
```

c04_dataloader_1.py

Simple DataLoader (1)

```python
# 3. Create and use the Dataset and DataLoader
if __name__ == '__main__':

    # Create a Dataset instance
    simple_dataset = SimpleDataset(my_data)

    # Create a DataLoader instance
    # Set the batch size to 8 and shuffle the data.
    data_loader = DataLoader(
        dataset=simple_dataset,
        batch_size=8,
        shuffle=True,
    )

    # Use the DataLoader to load data
    print(f"Total number of data points: {len(simple_dataset)}")
    print(f"Total number of batches: {len(data_loader)}")
    print(f"Batch size: {data_loader.batch_size}\n")

    for i, batch in enumerate(data_loader):
        print(f"--- Batch {i+1} ---")
        print(f"Batch tensor size: {batch.shape}")
        print(f"Data: {batch.tolist()}")

        # Print only the first 3 batches and then stop
        if i >= 2:
            break

    print("\nData loading complete!")
```

```
Total number of data points: 100
Total number of batches: 13
Batch size: 8

--- Batch 1 ---
Batch tensor size: torch.Size([8])
Data: [42.0, 61.0, 62.0, 55.0, 6.0, 54.0, 60.0, 47.0]
--- Batch 2 ---
Batch tensor size: torch.Size([8])
Data: [23.0, 7.0, 68.0, 80.0, 17.0, 37.0, 27.0, 50.0]
--- Batch 3 ---
Batch tensor size: torch.Size([8])
Data: [71.0, 66.0, 25.0, 75.0, 19.0, 41.0, 28.0, 4.0]

Data loading complete!
```

c04_dataloader_1.py

Simple DataLoader (2): Train/Val/Test Split

```python
import torch
from torch.utils.data import Dataset, DataLoader, random_split

# 1. Define a simple Dataset class
class SimpleDataset(Dataset):
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return torch.tensor(self.data[idx], dtype=torch.float32)


# 2. Prepare the data
my_data = list(range(100))  # numbers 0 to 99
simple_dataset = SimpleDataset(my_data)

# 3. Split into train, validation, and test sets
train_size = int(0.7 * len(simple_dataset))   # 70%
val_size   = int(0.2 * len(simple_dataset))   # 20%
test_size  = len(simple_dataset) - train_size - val_size  # 10%

train_set, val_set, test_set = random_split(simple_dataset, [train_size, val_size, test_size])
```

c04_dataloader_2.py

Simple DataLoader (2): Train/Val/Test Split

```python
# 4. Create DataLoaders
train_loader = DataLoader(train_set, batch_size=8, shuffle=True)
val_loader   = DataLoader(val_set, batch_size=8, shuffle=False)
test_loader  = DataLoader(test_set, batch_size=8, shuffle=False)

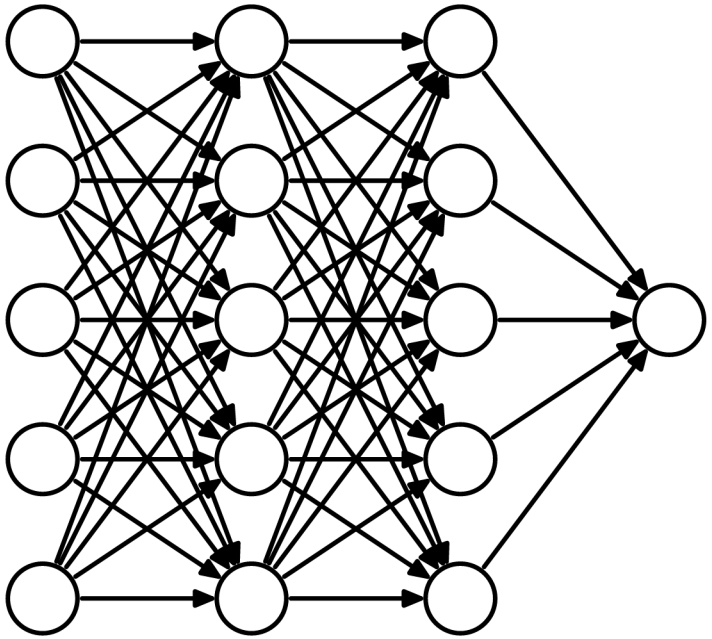
# 5. Example usage
if __name__ == '__main__':
    print(f"Dataset sizes -> Train: {len(train_set)}, Val: {len(val_set)}, Test: {len(test_set)}")

    for i, batch in enumerate(train_loader):
        print(f"Train Batch {i+1}: {batch.tolist()}")
        if i >= 2:     # Print only the first 3 batches and then stop
            break
```

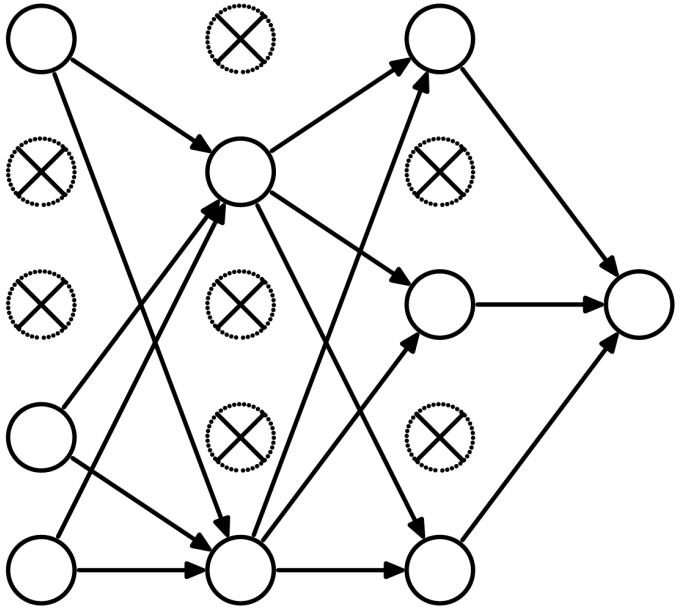
```
Dataset sizes -> Train: 70, Val: 20, Test: 10
Train Batch 1: [69.0, 26.0, 96.0, 76.0, 1.0, 72.0, 12.0, 29.0]
Train Batch 2: [14.0, 9.0, 87.0, 5.0, 44.0, 80.0, 34.0, 71.0]
Train Batch 3: [97.0, 65.0, 55.0, 3.0, 31.0, 48.0, 16.0, 63.0]
```

c04_dataloader_2.py

Dropout



(a) Standard Neural Net



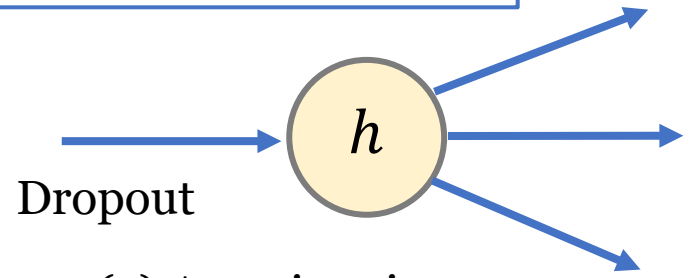
(b) After applying dropout.

Dropout Neural Net Model. **Left**: A standard neural net with 2 hidden layers. **Right**: An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

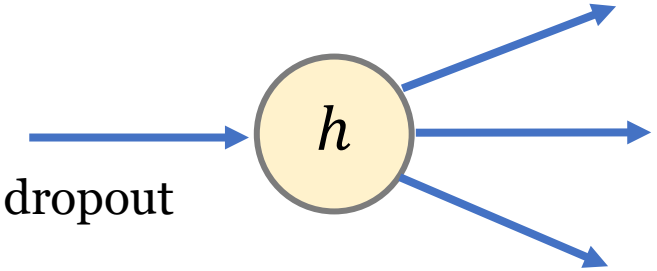
The key idea of **Dropout** is to ensure that the distribution of neuron outputs remains consistent between training and testing. Without such adjustment, the scale of outputs would differ, causing instability.

Definitions

- Dropout probability: p
- Neuron output before dropout: h



(a) At traing time



(b) At test time

Training Phase

- Probability that a neuron survives: $1 - p$
- If it survives: output is h
- If it is dropped: output is 0

Thus, the expected output during training is:

$$\mathbb{E}[h_{\text{train}}] = (1-p) \cdot h$$

mismatch

Testing Phase

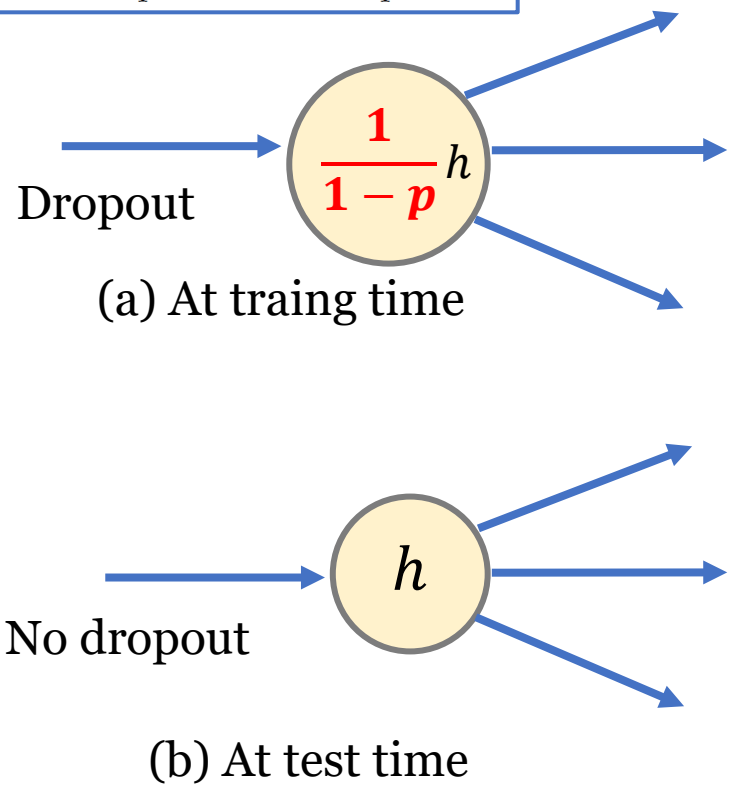
Since dropout is not used during testing, every neuron always produces its full output:

$$\mathbb{E}[h_{\text{test}}] = h$$

Inverted Dropout

Definitions

- Dropout probability: p
- Neuron output before dropout: h



Training Phase

Scaling Adjustment

Clearly, the expected outputs differ between training and testing. To fix this, we scale the active neuron outputs during training by a factor of $\frac{1}{1-p}$:

$$\mathbb{E}[h_{\text{scaled}}] = (1-p) \cdot \frac{h}{1-p} = h \tag{3}$$

This guarantees that the expected activations remain the same across both training and testing.

match

Testing Phase

Since dropout is not used during testing, every neuron always produces its full output:

$$\mathbb{E}[h_{\text{test}}] = h$$

By scaling in this way, we achieve consistent expected outputs, which leads to stable learning and inference. This method is known as **inverted dropout**.

Dropout

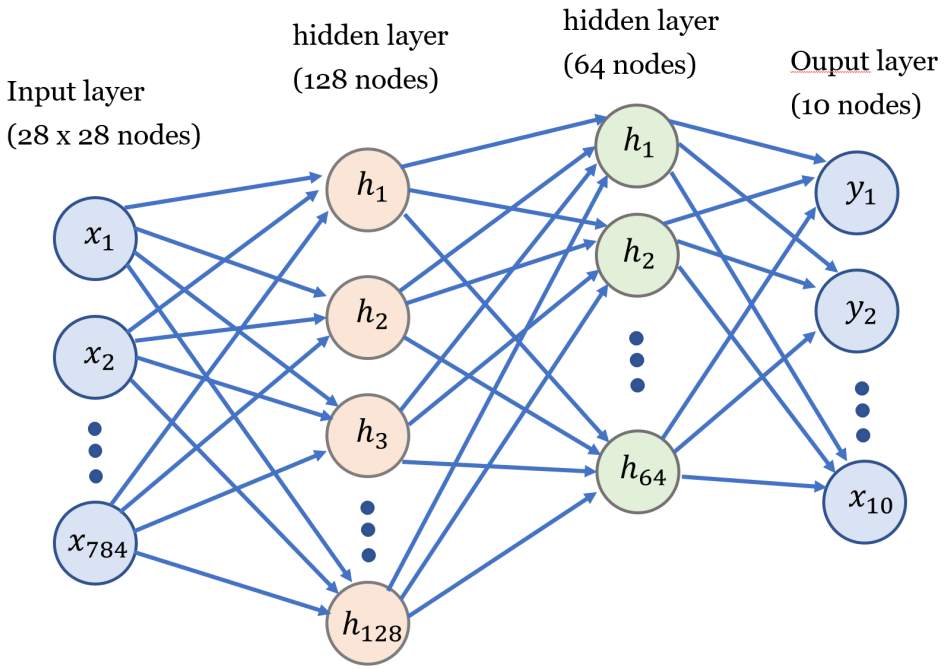
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# ====== Config ======
BATCH_SIZE = 128  # Use batches of 128 images
EPOCHS = 5        # Train for 5 epochs.
LR = 1e-3         # Dropout probability = 20%.
DROPOUT = 0.2
NUM_CLASSES = 10  # Output layer has 10 classes (digits 0–9).

# ====== Model ======
class SimpleNetMNIST(nn.Module):
    def __init__(self, p_drop=0.2):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.drop1 = nn.Dropout(p=p_drop)
        self.fc2 = nn.Linear(128, 64)
        self.drop2 = nn.Dropout(p=p_drop)
        self.fc3 = nn.Linear(64, NUM_CLASSES)

    def forward(self, x):
        x = x.view(x.size(0), -1)    # Flatten image: 28×28 = 784 pixels
        x = F.relu(self.fc1(x)); x = self.drop1(x)
        x = F.relu(self.fc2(x)); x = self.drop2(x)
        return self.fc3(x)           # return raw class scores
```

c04_dropout_1.py



```python
def main():
    torch.manual_seed(0)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # ----- Data (train only) -----
    transform = transforms.Compose([
        transforms.ToTensor(),     # Convert MNIST images to PyTorch tensors.
        transforms.Normalize((0.1307,), (0.3081,)) #Normalize using MNIST mean (0.1307) and std (0.3081).
    ])
    train_ds = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
    train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
                              num_workers=0, pin_memory=(device.type=="cuda"))

    # ----- Model / Loss / Optimizer -----
    model = SimpleNetMNIST(p_drop=DROPOUT).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LR)

    # ----- Train -----
    for ep in range(1, EPOCHS + 1):
        model.train()        # model.train() is a method defined in nn.Module.
        running, seen = 0.0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()
            bs = x.size(0)               # batch size (e.g., 128)
            running += loss.item() * bs  # add batch loss sum (<- loss.item() is already a mean, not the sum.)
            seen += bs                   # accumulate number of samples

        train_loss = running / max(1, seen) # This gives the true average loss across the whole epoch
        print(f"Epoch {ep:02d} | Train Loss {train_loss:.4f}")
```

```python
if __name__ == "__main__":
    main()
# raw logits
```

```
Epoch 01 | Train Loss 0.4140
Epoch 02 | Train Loss 0.1870
Epoch 03 | Train Loss 0.1457
Epoch 04 | Train Loss 0.1210
Epoch 05 | Train Loss 0.1096
```

c04_dropout_1.py

nn.Sequential ( )

```python
# ====== Model ======
class SimpleNetMNIST(nn.Module):
    def __init__(self, p_drop=0.2):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.drop1 = nn.Dropout(p=p_drop)
        self.fc2 = nn.Linear(128, 64)
        self.drop2 = nn.Dropout(p=p_drop)
        self.fc3 = nn.Linear(64, NUM_CLASSES)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x)); x = self.drop1(x)
        x = F.relu(self.fc2(x)); x = self.drop2(x)
        return self.fc3(x)
```

=

```python
# ====== Model ======
class SimpleNetMNIST(nn.Module):
    def __init__(self, p_drop=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(64, NUM_CLASSES)  # last layer has no ReLU
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)  # flatten
        return self.net(x)
```

nn.Sequential

is a convenience **container** that lets you stack **layers in order and run them one after another.**

It's a **subclass of nn.Module**, so it behaves like any other model: you can move it to GPU/CPU, call .train()/.eval(), save/load state_dict, etc.

nn.Sequential ( )

Advantages

- Code is **shorter** and easier to read.
- Convenient **when the layer order is simple.**

Disadvantages

- **Harder** to add custom operations in the forward pass (e.g., merging multiple inputs, **skip connections**).
- For research or more complex experiments, writing your own **forward( )** is more flexible

transforms.Compose( )

```
from torchvision import datasets, transforms

# Define a transformation pipeline
transform = transforms.Compose([
    transforms.ToTensor(),                        # Convert PIL image (H x W x C) in [0,255] → Tensor (C x H x W) in [0.0,1.0]
    transforms.Normalize((0.1307,), (0.3081,))  # Normalize with mean and std (for MNIST)
])

# Apply to dataset
train_ds = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
```

- transforms.Compose = “make a pipeline of transforms, run them in sequence on every sample.”

transforms.Compose([...]) lets you **combine multiple steps into one transform object**.

When the dataset calls __getitem__, the image is automatically passed through all these steps before being returned.

Data Augmentation (CIFAR-10)

Using transforms.Compose( )

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),   # randomly flip left-right
    transforms.RandomCrop(32, padding=4), # random crop with padding
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # normalize per channel
])

train_ds = datasets.CIFAR10(root="./data", train=True, download=True, transform=transform_train)
```

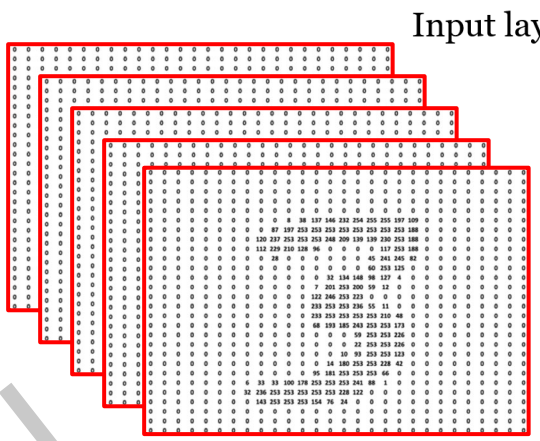
transforms.Compose( )

- It's a **wrapper** that allows you to **chain multiple image transformations together.**
- **Each transformation is applied in order to the input data (usually an image).**
- Commonly used with datasets in torchvision.datasets (e.g., MNIST, CIFAR-10).
- Think of it like a **pipeline**: the image passes through one transform, then the next, and so on.

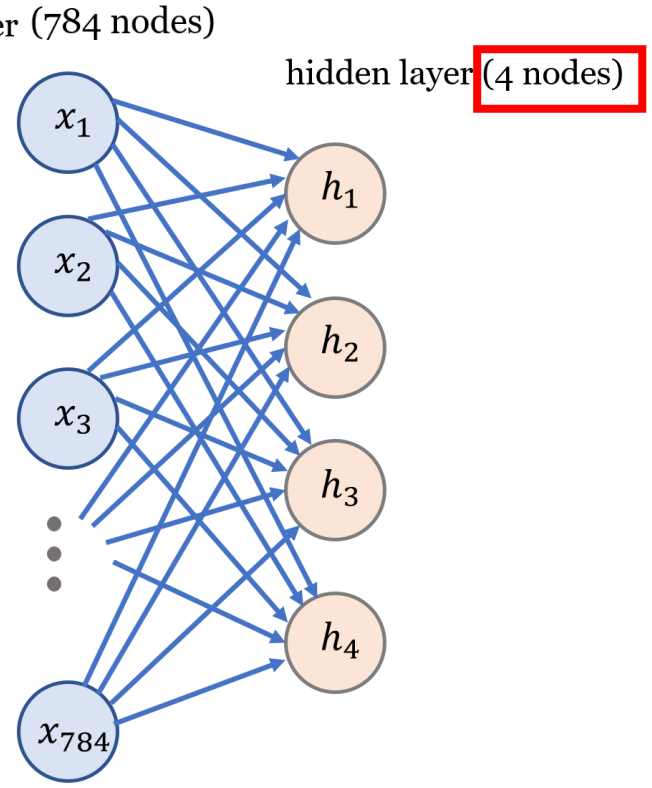
Concept	Meaning	Equivalent Term
Batch size	Number of samples in the mini-batch	N (rows)
Hidden dimension	Number of nodes in a hidden layer	Number of features (m, columns)
Feature	One value from a hidden layer node	Same as hidden node output

Example 1





- Batch size = 5



- Number of features in a hidden node (= dimension of hidden node= number of features in a hidden node) = 4

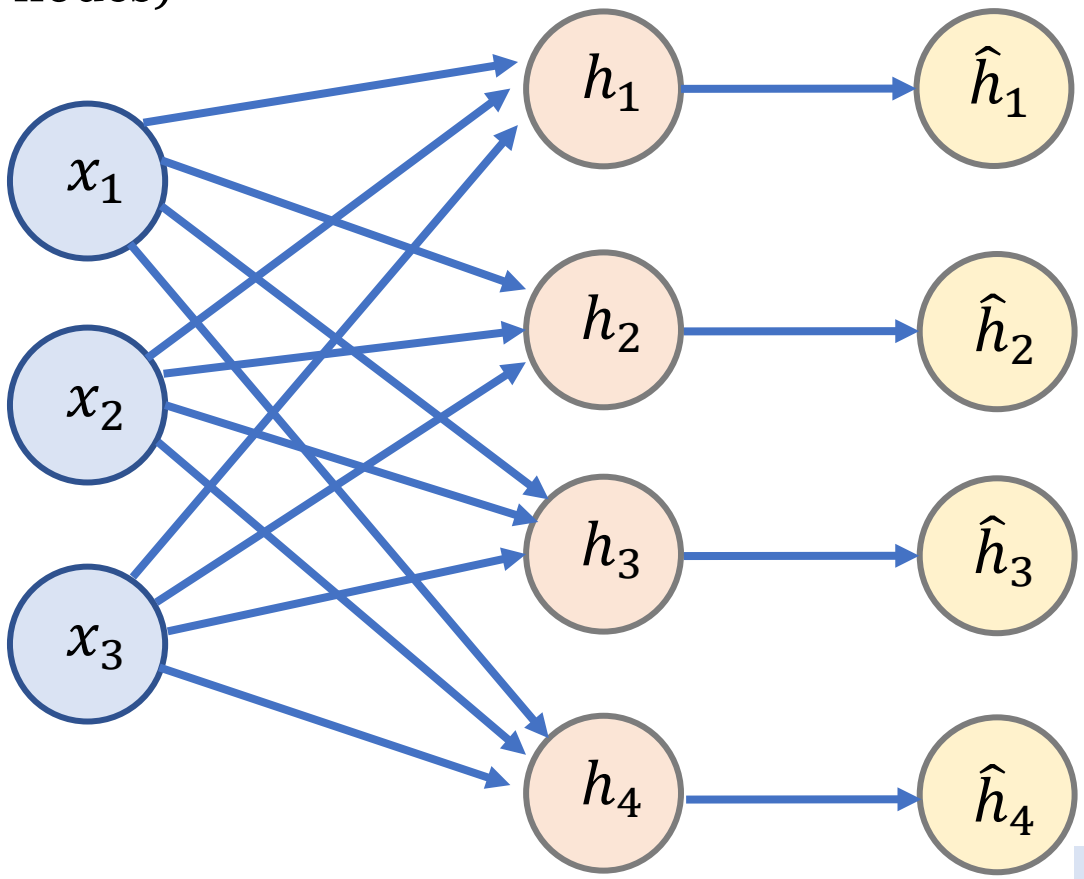
Batch Normalization (BN)

Example 2

Input layer
(3 nodes)

hidden layer
(4 nodes)

$\hat{\gamma}_1, \hat{\beta}_1$



- This stabilizes training and **allows higher learning rates**. (reduce training time and performance)
- The performance of BN depends on the mini-batch size (disadvantage).
- Works well for **CNNs** (convolutional neural networks).

Batch Normalization (per feature, across batch):

$$\hat{h}_{ij} = \frac{h_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \cdot \hat{\gamma}_j + \hat{\beta}_j \tag{1}$$

where

$$\mu_j = \frac{1}{N} \sum_i h_{ij} \quad \text{(batch mean of feature } j\text{)}$$

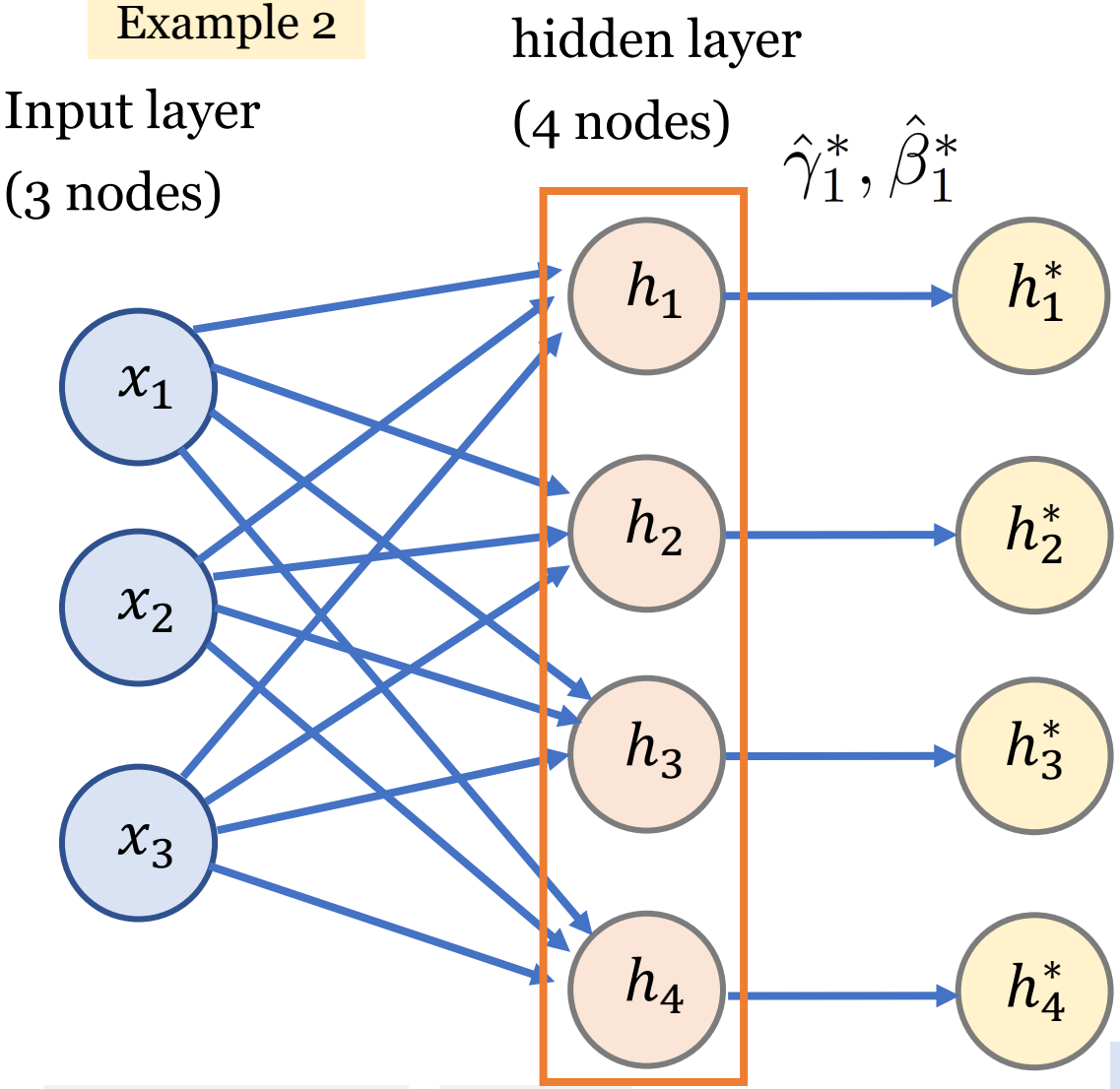
$$\sigma_j^2 = \frac{1}{N} \sum_i (h_{ij} - \mu_j)^2 \quad \text{(batch variance of feature } j\text{)}$$

Here, i is the sample index and j is the feature index. $\hat{\gamma}_j$ and $\hat{\beta}_j$ are learnable parameters (scale and shift for each feature).

BN: Normalizes *across the batch* → depends on other samples in the mini-batch.

Layer Normalization (LN)

Example 2



- This stabilizes training and **allows higher learning rates**. (reduce training time and performance)
- The performance of LN does **NOT** depend on the mini-batch size (advantage).
- Works well for RNNs /**Transformer** where **batch sizes change or are small**.

Layer Normalization (per sample, across features):

$$h_{ij}^* = \frac{h_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \cdot \gamma_j^* + \beta_j^* \tag{2}$$

where

$$\mu_i = \frac{1}{D}\sum_j h_{ij} \quad \text{(feature mean of sample } i\text{)}$$

$$\sigma_i^2 = \frac{1}{D}\sum_j (h_{ij} - \mu_i)^2 \quad \text{(feature variance of sample } i\text{)}$$

Here, D is the feature dimension size. Each feature j has its own learnable parameters γ_j^* and β_j^*.

LN: Normalizes *per sample* → independent on other samples

Batch Normalization (BN)

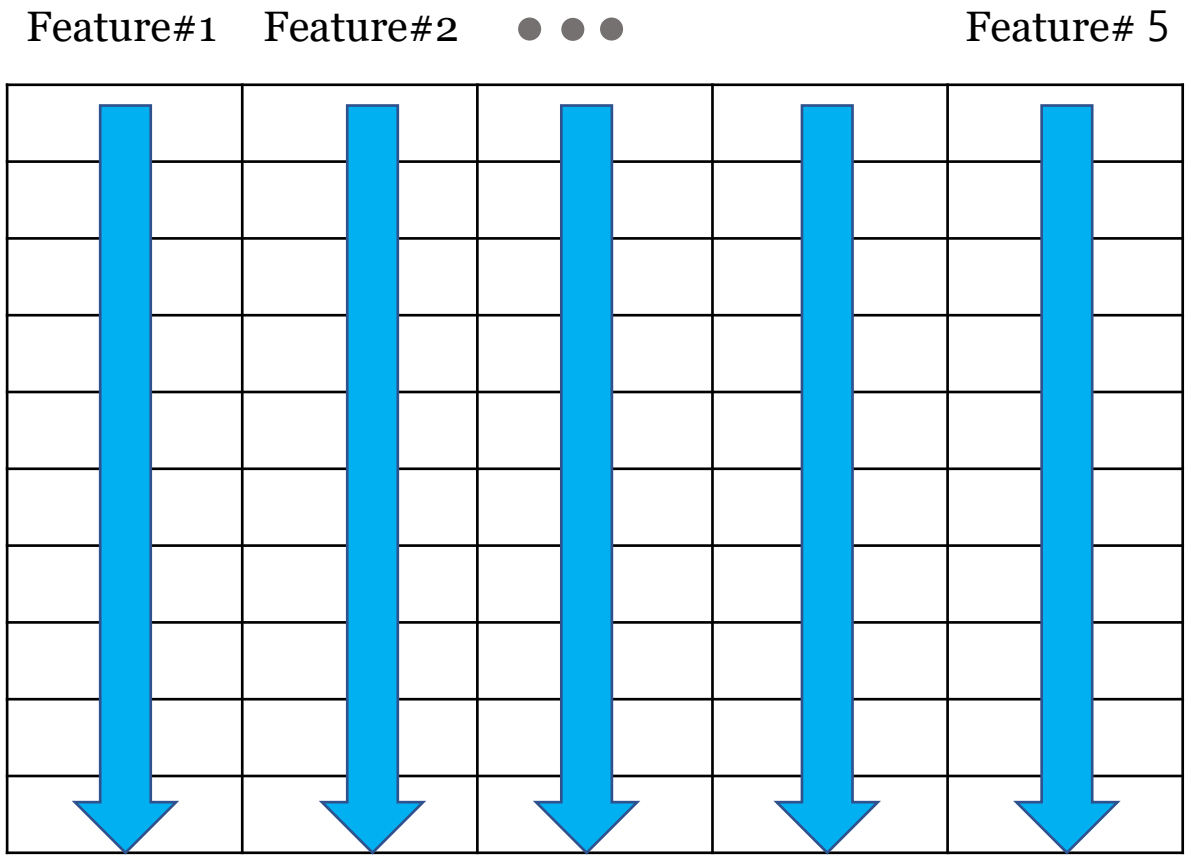
$\widehat{h}_1$

$\hat{\gamma}_1, \hat{\beta}_1$

Example 3

- Batch size = 10 (10 samples in the batch)
- Hidden layer dimension = 5 (each hidden layer output vector has 5 nodes/features)

Thus, the input tensor shape is $[10, 5]$: 10 rows (samples) × 5 columns (features).



Batch Normalization computes statistics **per feature** across the batch.

- For each of the 5 features, compute the mean and variance across all 10 samples.
- Mean: 5 values (one per feature)
- Variance: 5 values (one per feature)

Therefore, the total number of (mean, std) pairs is:

$$\text{Total pairs (BN)} = 5 \quad (= \text{hidden dimension}) \qquad (1)$$

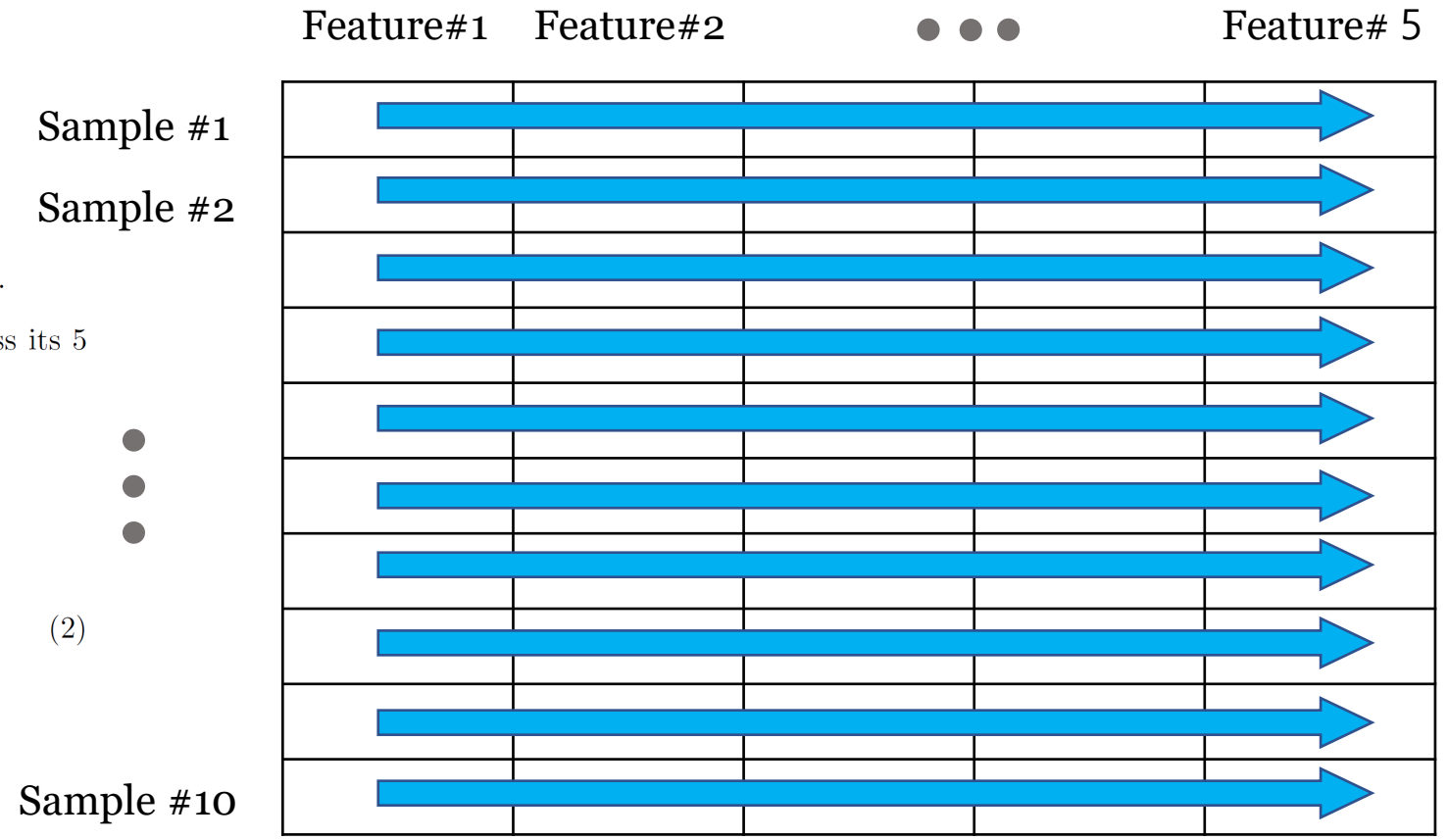
Layer Normalization (LN)

Example 3

- Batch size = 10 (10 samples in the batch)
- Hidden layer dimension = 5 (each hidden layer output vector has 5 nodes/features)

Thus, the input tensor shape is $[10, 5]$: 10 rows (samples) $\times$ 5 columns (features).

$\hat{\gamma}_1^*, \hat{\beta}_1^*$ h_1^*



Layer Normalization computes statistics **per sample** across its features.

- For each of the 10 samples, compute the mean and variance across its 5 features.
- Mean: 10 values (one per sample)
- Variance: 10 values (one per sample)

Therefore, the total number of (mean, std) pairs is:

$$\text{Total pairs (LN)} = 10 \quad (= \text{batch size}) \tag{2}$$

1 Standard Order in Practice

In most deep learning architectures, the common sequence is:

$$\text{(Linear / Convolution)} \rightarrow \text{Normalization (BN or LN)} \rightarrow \text{Activation (ReLU, GELU, etc.)} \tag{1}$$

This ordering is used in ResNets, Transformers, and many other models.

2 Why Normalization Comes Before Activation

2.1 Normalization stabilizes statistics

Batch Normalization (BN) and Layer Normalization (LN) normalize the distribution of the pre-activation values (mean ≈ 0, variance ≈ 1). This ensures that the input to the nonlinear activation is in a stable and well-scaled range.

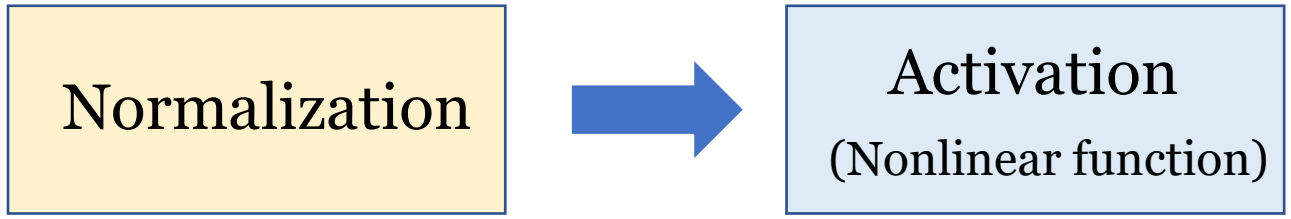
2.2 Activation functions skew distributions

If you apply activation before normalization, then the input distribution to normalization is already distorted (e.g., ReLU truncates negative values to zero). This breaks the assumption that the normalized features are drawn from a reasonably symmetric distribution, making normalization less effective.

2.3 Gradient flow

BN/LN provide smoother gradient propagation if applied before the nonlinearity. If applied after, the nonlinearity (e.g., ReLU's zeroing effect) may destroy variance information, which normalization cannot recover.

Batch Normalization vs Layer Normalization Order





- **Normalization before activation** → maintains stable statistics, improves optimization, better gradient flow.



- **Normalization after activation** → distribution is skewed, variance reduced, harder optimization, degraded performance.

The theoretical basis lies in **distributional stability** and **gradient flow preservation**.

3 Theoretical Justification

3.1 Variance preservation

Normalization layers ensure unit variance. If placed after an activation like ReLU, which already reduces variance (by half in expectation), normalization may counteract this, but in doing so it alters the intended effect of the activation. This leads to unstable representations.

3.2 Covariate shift reduction

One of BN's original motivations is to reduce internal covariate shift (changes in the distribution of hidden activations). If applied after nonlinearities, BN/LN no longer stabilize the inputs to those nonlinearities—so they cannot serve their main theoretical role.

3.3 Empirical results

Many studies (e.g., the original BN paper, later Transformer papers on LN) show worse convergence and accuracy when swapping the order. For example, "BN → ReLU" is consistently better than "ReLU → BN" in CNNs. Similarly, Transformers always use "LN → Attention/MLP → Residual", not the reverse.

Batch Normalization (BN)

Example: hidden layer dimension = 128
→ 128 nodes in the hidden layer
→ 128 features in the hidden representation

Example 4

```python
import torch
import torch.nn as nn

# (Example) batch of 32 samples, each with
# 128 features
x = torch.randn(32, 128)

# -------------------------
# Batch Normalization (using nn.BatchNorm1d( ))
# -------------------------
bn = nn.BatchNorm1d(128)   # 128 features
out_bn = bn(x)

print("BN output shape:", out_bn.shape)
```

```
BN output shape: torch.Size([32, 128])
```

C04_BN_LN_1a.py

=

```python
import torch

# (Example) batch of 32 samples, each with 128 features
x = torch.randn(32, 128)

eps = 1e-5  # small constant for numerical stability

# -------------------------
# Batch Normalization (manual)
# -------------------------
mean_bn = x.mean(dim=0, keepdim=True)          # [1, 128]
var_bn = x.var(dim=0, unbiased=False, keepdim=True) # [1, 128]
x_bn = (x - mean_bn) / torch.sqrt(var_bn + eps)

# learnable parameters (gamma, beta)
gamma_bn = torch.ones(1, 128)
beta_bn = torch.zeros(1, 128)

out_bn = gamma_bn * x_bn + beta_bn

print("BN output shape:", out_bn.shape)
```

C04_BN_LN_1b.py

```
BN output shape: torch.Size([32, 128])
```

- Compute the mean and variance along the **batch dimension** (dim=0).
- This gives statistics of shape $[1, 128]$, i.e., one mean and variance per feature.
- Normalize: $(x - \mu)/\sqrt{\sigma^2 + \epsilon}$.
- Apply learnable parameters γ (scale) and β (shift), each of shape $[1, 128]$.

Layer Normalization (LN)

Example: hidden layer dimension = 128
→ 128 nodes in the hidden layer
→ 128 features in the hidden representation

Example 4

```python
import torch
import torch.nn as nn

# (Example) batch of 32 samples, each with
# 128 features
x = torch.randn(32, 128)

# -------------------------
# Layer Normalization (using nn.LayerNorm( ))
# -------------------------
ln = nn.LayerNorm(128)     # 128 features
out_ln = ln(x)

print("LN output shape:", out_ln.shape)
```

```
LN output shape: torch.Size([32, 128])
```

C04_BN_LN_1a.py

=

```python
import torch

# (Example) batch of 32 samples, each with 128 features
x = torch.randn(32, 128)

eps = 1e-5  # small constant for numerical stability

# -------------------------
# Layer Normalization (manual)
# -------------------------
mean_ln = x.mean(dim=1, keepdim=True)          # [32, 1]
var_ln = x.var(dim=1, unbiased=False, keepdim=True)  # [32, 1]
x_ln = (x - mean_ln) / torch.sqrt(var_ln + eps)

# learnable parameters (gamma, beta)
gamma_ln = torch.ones(1, 128)   # broadcast across batch
beta_ln = torch.zeros(1, 128)
out_ln = gamma_ln * x_ln + beta_ln

print("LN output shape:", out_ln.shape)
```

```
LN output shape: torch.Size([32, 128])
```

C04_BN_LN_1b.py

- Compute the mean and variance along the **feature dimension** for each sample (dim=1).
- This gives statistics of shape $[32, 1]$, i.e., one mean and variance per sample.
- Normalize: $(x - \mu)/\sqrt{\sigma^2 + \epsilon}$.
- Apply learnable parameters γ and β, each of shape $[1, 128]$, broadcast across samples.

Summary

Dropout

- `model.train()` → randomly disables some neurons (for regularization).
- `model.eval()` → disables dropout, all neurons are active.

BatchNorm

- `model.train()` → uses batch statistics (mean/variance of the current mini-batch).
- `model.eval()` → uses the running mean/variance tracked during training.

`C04_BN_LN_1b.py`

Thank you!

